



TRADINGPRO

Bilag 3 - Test Rapport

Ozan Korkmaz

Vejleder: Flemming Sørensen

1205hf26086per

03-09-2026

Inholdsfortegnelse

1. Formål	3
2. Teststrategi	3
3. Samlet resultat	3
4. Testcases med forklaring	3
5. Komplet testkode	5
6. Begrænsninger.....	18

1. Formål

Dette bilag dokumenterer de automatiserede tests i TradingAppTest. Der er 39 automatiserede tests: 21 repository-tests og 18 controller-tests. Den seneste dokumenterede testkørsel bestod alle 39: 39 bestået og 0 fejlet.

2. Teststrategi

Repository-tests bruger EF Core InMemory. Controller-tests bruger mocks eller den relevante service/repository-kæde afhængigt af controllerens afhængigheder. Exchange-controllerens tests går gennem ExchangeManagementService og ExchangeRepository, så den nye lagdeling også indgår i testopsætningen. Testene følger Arrange-Act-Assert: først opstilles data og mocks, derefter udføres handlingen, og til sidst kontrolleres resultatet.

3. Samlet resultat

Område	Antal	Resultat
Repositories	21	21/21 bestået
Controllers	18	18/18 bestået
I alt	39	39/39 bestået - 0 fejlet

4. Testcases med forklaring

Controllers/CandlesControllerTests.cs

Tester CandlesControllerens validering, limit-beskyttelse og normale succesflow for historical candle-requests.

#	Testmetode	Kort forklaring	Resultat
1	GetCandles_InvalidSymbolId_ReturnsBadRequest	Kontrollerer inputvalideringen i CandlesController. Hvis symbolId er ugyldigt, skal requesten stoppes tidligt og returnere HTTP 400 Bad Request.	Bestået
2	GetCandles_ClampsLimitTo1000	Kontrollerer at en for stor limit bliver begrænset til 1000, før requesten sendes videre. Det beskytter API og database mod unødvendigt store forespørgsler.	Bestået
3	GetCandles_ValidRequest_ReturnsServiceResult	Kontrollerer normal succesadfærd: en gyldig candle-request sendes til servicen, og controlleren returnerer det forventede resultat.	Bestået

Controllers/ExchangesControllerTests.cs

Tester administration af exchanges: aktive exchanges, oprettelse, adapterkontrol, dubletter, opdatering og sikker sletning.

#	Testmetode	Kort forklaring	Resultat
4	GetActiveExchanges_ReturnsOnlyActiveRows	Kontrollerer at endpointet kun viser aktive exchanges og ikke poster, som er deaktiveret eller slettet.	Bestået
5	CreateExchange_WithSupportedAdapter_CreatesNormalizedExchange	Kontrollerer at en exchange med en eksisterende backend-adapter kan oprettes, og at exchange-koden normaliseres før lagring.	Bestået
6	CreateExchange_WithoutBackendAdapter_ReturnsBadRequest	Kontrollerer at en exchange ikke kan aktiveres alene ved at oprette en databasepost. Der skal også findes en implementeret IExchangeService-adapter.	Bestået
7	CreateExchange_WhenCodeAlreadyExists_ReturnsConflict	Kontrollerer at samme exchange-kode ikke kan oprettes to gange. Ved dublet returneres HTTP 409 Conflict.	Bestået
8	UpdateExchange_CannotChangeCodeAfterSymbolsExist	Kontrollerer at exchange-koden ikke kan ændres, når der allerede findes symbols på exchange. Det beskytter eksisterende market-data og relationer mod at blive inkonsistente.	Bestået
9	DeleteExchange_WithSymbols_ReturnsConflictAndKeepsExchange	Kontrollerer at en exchange med eksisterende symbols ikke slettes. Controlleren skal returnere HTTP 409 Conflict og beholde exchange-posten.	Bestået
10	DeleteExchange_WithoutSymbols_ReturnsNoContent	Kontrollerer succesflowet ved sletning af en exchange uden symbols. Handlingen gennemføres, og API'et returnerer HTTP 204 No Content.	Bestået

Controllers/SymbolsControllerTests.cs

Tester symbol-endpoints, herunder exchange-id validering, opslag, manuel sync og statusændringer.

#	Testmetode	Kort forklaring	Resultat
11	GetSymbols_InvalidExchangeId_ReturnsBadRequest	Kontrollerer at et ugyldigt exchangeid bliver afvist med HTTP 400, før symbolsøgningen udføres.	Bestået
12	GetSymbol_WhenMissing_ReturnsNotFound	Kontrollerer at opslag på et symbol, der ikke findes, returnerer HTTP 404 i stedet for et tomt succesresultat.	Bestået
13	SyncSymbols_NormalizesExchangeCodeAndCallsSyncService	Kontrollerer at exchange-koden normaliseres, og at den korrekte sync-service bliver kaldt ved manuel symbolsynkronisering.	Bestået
14	ToggleStatus_WhenSymbolMissing_ReturnsNotFound	Kontrollerer at aktivering/deaktivering af et ikke-eksisterende symbol returnerer HTTP 404.	Bestået

Controllers/WatchlistsControllerTests.cs

Tester watchlist-API'et med fokus på autentificeret bruger, inputvalidering og korrekt HTTP-resultat ved tilføjelse/fjernelse.

#	Testmetode	Kort forklaring	Resultat
15	GetMyWatchlists_UsesAuthenticatedUserId	Kontrollerer at watchlists hentes for den autentificerede bruger og ikke ud fra et frit bruger-id sendt fra klienten.	Bestået
16	CreateWatchlist_BlankName_ReturnsBadRequest	Kontrollerer at en watchlist uden et gyldigt navn afvises med HTTP 400.	Bestået
17	AddSymbolToWatchlist_InvalidIds_ReturnsBadRequest	Kontrollerer at ugyldige watchlist- eller symbol-id'er afvises, før service- og databaseoperationer udføres.	Bestået
18	RemoveSymbolFromWatchlist_WhenServiceReturnsFalse_ReturnsNotFound	Kontrollerer at API'et returnerer HTTP 404, når det ønskede symbol/watchlist-item ikke kan findes og fjernes.	Bestået

Repositories/CandleRepositoryTests.cs

Tester repository-logik for candle-rækkefølge, lazy loading med endTime og update-vs-duplicate adfærd.

#	Testmetode	Kort forklaring	Resultat
19	GetCandlesAsync_OneMinute_ReturnsLatestLimitInAscendingOrder	Kontrollerer første indlæsning af 1m candles. Repository'et vælger de seneste N candles og returnerer dem i stigende OpenTime, så chartet får korrekt kronologisk rækkefølge.	Bestået
20	GetCandlesAsync_EndTime_ReturnsOnlyCandlesBeforeEndTime	Kontrollerer lazy loading med endTime. Kun candles før cursortidspunktet må returneres, så frontend kan hente ældre historik ved navigation mod venstre.	Bestået
21	AddOrUpdateCandleAsync_UpdatesExistingCandleInsteadOfDuplicating	Kontrollerer upsert-adfærden. Hvis samme candle allerede findes, opdateres OHLCV-værdierne i stedet for at indsætte en dublet.	Bestået

Repositories/SymbolRepositoryTests.cs

Tester normalisering, aktive symbols, exchange-filtrering og case-insensitive symbolopslag.

#	Testmetode	Kort forklaring	Resultat
22	AddAsync_NormalizesSymbolFields	Kontrollerer at symbol, base asset og quote asset normaliseres før lagring. Det sikrer ensartede værdier på tværs af API, database og realtime-grupper.	Bestået
23	GetAllActiveSymbolsAsync_ReturnsOnlyActiveSymbolsFromActiveExchange	Kontrollerer at kun aktive symbols fra aktive exchanges returneres til market-data processen.	Bestået
24	SearchAsync_WithExchangeId_DoesNotLeakSymbolsFromOtherExchange	Kontrollerer exchange-isolation ved symbolsøgning. En søgning for én exchange må ikke returnere matches fra en anden exchange.	Bestået
25	GetByNameAsync_IsCaseInsensitiveAndTrimsInput	Kontrollerer at symbolopslag ignorerer store/små bogstaver og ekstra mellemrum i input.	Bestået

Repositories/UserRepositoryTests.cs

Tester login-opslag, soft delete, preferences-upsert og filtrering af slettede brugere.

#	Testmetode	Kort forklaring	Resultat
26	GetByLoginIdentifierAsync_FindsByUsernameOrEmailCaseInsensitively	Kontrollerer at login-opslag kan finde brugeren via både username og e-mail uden at være afhængig af store/små bogstaver.	Bestået
27	DeleteAsync_PerformsSoftDelete	Kontrollerer at brugersletning udføres som soft delete, så posten bevares i databasen men ikke behandles som en aktiv bruger.	Bestået
28	SavePreferencesAsync_UpsertsPreferences	Kontrollerer at brugerindstillinger oprettes første gang og opdateres ved efterfølgende gemninger i stedet for at skabe dubletter.	Bestået
29	GetAllUsersAsync_ExcludesSoftDeletedUsers	Kontrollerer at soft-deleted users ikke kommer med i den normale brugeroversigt.	Bestået

Repositories/WatchlistRepositoryTests.cs

Tester indlæsning af watchlist-relationer, fjernelse af enkelte symbols og sletning af tilhørende items.

#	Testmetode	Kort forklaring	Resultat
30	GetWatchlistsByUserIdAsync_LoadsItemsSymbolAndExchange	Kontrollerer at en watchlist indlæses med de nødvendige relationer: items, symbol og exchange, så frontend får et komplet resultat.	Bestået
31	RemoveItemAsync_RemovesOnlyRequestedSymbol	Kontrollerer at fjernelse af ét symbol fra en watchlist ikke påvirker de øvrige symbols i samme liste.	Bestået
32	DeleteAsync_CascadeDeletesWatchlistItems	Kontrollerer at watchlist-items fjernes sammen med watchlisten, så der ikke efterlades forældreløse relationer.	Bestået

Repositories/ExchangeRepositoryTests.cs

Tester repository-logik for aktive exchanges, id-opslag, duplicate code, symbolrelationer samt oprettelse, opdatering og sletning.

#	Testmetode	Kort forklaring	Resultat
33	GetActiveAsync_ReturnsOnlyActiveExchangesOrderedByName	Kontrollerer at kun aktive exchanges returneres, og at resultatet sorteres efter navn.	Bestået
34	GetByIdAsync_ExistingExchange_ReturnsExchange	Kontrollerer at en eksisterende exchange kan findes korrekt via id.	Bestået
35	ExistsByCodeAsync_RespectsExcludedExchangeId	Kontrollerer duplicate-code opslag og at exclude-id virker korrekt ved opdatering.	Bestået
36	HasSymbolsAsync_ReturnsTrueOnlyWhenSymbolsExist	Kontrollerer om repository'et korrekt registrerer tilknyttede symbols på en exchange.	Bestået
37	AddAsync_PersistsExchange	Kontrollerer at en ny exchange gemmes i databasen via repository'et.	Bestået
38	SaveChangesAsync_PersistsUpdatedExchange	Kontrollerer at ændringer i navn og aktiv status gemmes korrekt.	Bestået
39	DeleteAsync_RemovesExchange	Kontrollerer at en exchange uden tilknyttede data kan fjernes fra databasen.	Bestået

5. Komplet testkode

Nedenfor gengives testkoden som dokumentation for de beskrevne testscenarier.

Controllers/CandlesControllerTests.cs

Tester CandlesControllerens validering, limit-beskyttelse og normale succesflow for historical candle-requests.

```
using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
```

```
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class CandlesControllerTests
{
    // Kontrollerer, at et ugyldigt symbol-id bliver afvist med Bad Request.
    [Fact]
    public async Task GetCandles_InvalidSymbolId_ReturnsBadRequest()
    {
        var controller = new CandlesController(Mock.Of<ICandleService>());

        var result = await controller.GetCandles(0, "1m", 100, null);

        Assert.IsType<BadRequestObjectResult>(result.Result);
    }

    // Kontrollerer, at en for stor candle-limit bliver begrænset før servicekaldet.
    [Fact]
    public async Task GetCandles_ClampsLimitTo1000()
    {
        var service = new Mock<ICandleService>();
        service.Setup(x => x.GetCandlesAsync(1, "15m", 1000, null))
            .ReturnsAsync(Array.Empty<CandleResponseDto>());
        var controller = new CandlesController(service.Object);

        var result = await controller.GetCandles(1, "15m", 5000, null);

        Assert.IsType<OkObjectResult>(result.Result);
        service.Verify(x => x.GetCandlesAsync(1, "15m", 1000, null), Times.Once);
    }

    // Kontrollerer, at en gyldig request returnerer det forventede service-resultat.
    [Fact]
    public async Task GetCandles_ValidRequest_ReturnsServiceResult()
    {
        var expected = new[]
        {
            new CandleResponseDto(1, 100, 110, 90, 105, 5, 1, "BTCUSDT", "BINANCE", "1m")
        };
        var service = new Mock<ICandleService>();
        service.Setup(x => x.GetCandlesAsync(1, "1m", 100, null)).ReturnsAsync(expected);
        var controller = new CandlesController(service.Object);

        var result = await controller.GetCandles(1, "1m", 100, null);

        var ok = Assert.IsType<OkObjectResult>(result.Result);
        Assert.Same(expected, ok.Value);
    }
}
```

Controllers/ExchangesControllerTests.cs

Tester administration af exchanges: aktive exchanges, oprettelse, adapterkontrol, dubletter, opdatering og sikker sletning.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppLibrary.Services;
using TradingAppTest.Helpers;

namespace TradingAppTest.Controllers;

public class ExchangesControllerTests
{
    // Kontrollerer, at kun aktive exchanges returneres.
    [Fact]
    public async Task GetActiveExchanges_ReturnsOnlyActiveRows()
    {
        await using var db = TestDbContextFactory.Create();
    }
}
```

```
db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
await db.SaveChangesAsync();
var controller = CreateController(db);

var result = await controller.GetActiveExchanges(CancellationToken.None);

var ok = Assert.IsType<OkObjectResult>(result.Result);
var exchanges = Assert.IsAssignableFrom<IEnumerable<ExchangeResponseDto>>(ok.Value).ToList();
var exchange = Assert.Single(exchanges);
Assert.Equal("BINANCE", exchange.Code);
}

// Kontrollerer, at en exchange med en understøttet adapter kan oprettes.
[Fact]
public async Task CreateExchange_WithSupportedAdapter_CreatesNormalizedExchange()
{
    await using var db = TestDbContextFactory.Create();
    var controller = CreateController(db);

    var result = await controller.CreateExchange(
        new CreateExchangeDto(" okx ", " OKX Exchange ", true), CancellationToken.None);

    var created = Assert.IsType<CreatedAtActionResult>(result.Result);
    var dto = Assert.IsType<ExchangeResponseDto>(created.Value);
    Assert.Equal("OKX", dto.Code);
    Assert.Equal("OKX Exchange", dto.Name);
    Assert.True(await db.Exchanges.AnyAsync(e => e.Code == "OKX"));
}

// Kontrollerer, at en exchange uden backend-adapter bliver afvist.
[Fact]
public async Task CreateExchange_WithoutBackendAdapter_ReturnsBadRequest()
{
    await using var db = TestDbContextFactory.Create();
    var controller = CreateController(db);

    var result = await controller.CreateExchange(
        new CreateExchangeDto("KRAKEN", "Kraken", true), CancellationToken.None);

    Assert.IsType<BadRequestObjectResult>(result.Result);
    Assert.False(await db.Exchanges.AnyAsync(e => e.Code == "KRAKEN"));
}

// Kontrollerer, at en eksisterende exchange-kode ikke kan oprettes igen.
[Fact]
public async Task CreateExchange_WhenCodeAlreadyExists_ReturnsConflict()
{
    await using var db = TestDbContextFactory.Create();
    var controller = CreateController(db);

    var result = await controller.CreateExchange(
        new CreateExchangeDto("BINANCE", "Another Binance", true), CancellationToken.None);

    Assert.IsType<ConflictObjectResult>(result.Result);
}

// Kontrollerer, at exchange-koden ikke kan ændres, når der allerede findes symboler.
[Fact]
public async Task UpdateExchange_CannotChangeCodeAfterSymbolsExist()
{
    await using var db = TestDbContextFactory.Create();
    db.Symbols.Add(new Symbol { Exchangeld = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT"
});
```

```

        await db.SaveChangesAsync();
        var controller = CreateController(db);

        var result = await controller.UpdateExchange(
            1, new UpdateExchangeDto("OKX", "Changed", true), CancellationToken.None);

        Assert.IsType<ConflictObjectResult>(result.Result);
        Assert.Equal("BINANCE", (await db.Exchanges.FindAsync(1))!.Code);
    }

    // Kontrollerer, at en exchange med eksisterende symboler ikke kan slettes.
    [Fact]
    public async Task DeleteExchange_WithSymbols_ReturnsConflictAndKeepsExchange()
    {
        await using var db = TestDbContextFactory.Create();
        db.Symbols.Add(new Symbol { Exchangeld = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT"
    });
        await db.SaveChangesAsync();
        var controller = CreateController(db);

        var result = await controller.DeleteExchange(1, CancellationToken.None);

        Assert.IsType<ConflictObjectResult>(result);
        Assert.NotNull(await db.Exchanges.FindAsync(1));
    }

    // Kontrollerer, at en exchange uden symboler kan slettes korrekt.
    [Fact]
    public async Task DeleteExchange_WithoutSymbols_ReturnsNoContent()
    {
        await using var db = TestDbContextFactory.Create();
        db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
        await db.SaveChangesAsync();
        var controller = CreateController(db);

        var result = await controller.DeleteExchange(2, CancellationToken.None);

        Assert.IsType<NoContentResult>(result);
        Assert.Null(await db.Exchanges.FindAsync(2));
    }

    private static ExchangesController CreateController(TradingAppLibrary.Data.AppDbContext db)
    {
        var repository = new ExchangeRepository(db);
        var service = new ExchangeManagementService(
            repository,
            new FakeExchangeServiceFactory("BINANCE", "OKX"));

        return new ExchangesController(service);
    }
}

```

Controllers/SymbolsControllerTests.cs

Tester symbol-endpoints, herunder exchange-id validering, opslag, manuel sync og statusændringer.

```

using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class SymbolsControllerTests

```



```

{
    // Kontrollerer, at et ugyldigt exchange-id bliver afvist.
    [Fact]
    public async Task GetSymbols_InvalidExchangeId_ReturnsBadRequest()
    {
        var controller = new SymbolsController(Mock.Of<ISymbolService>(), Mock.Of<IMarketDataSyncService>());

        var result = await controller.GetSymbols(exchangeId: 0);

        Assert.IsType<BadRequestObjectResult>(result.Result);
    }

    // Kontrollerer, at et manglende element returnerer Not Found.
    [Fact]
    public async Task GetSymbol_WhenMissing_ReturnsNotFound()
    {
        var symbols = new Mock<ISymbolService>();
        symbols.Setup(x => x.GetSymbolByIdAsync(99)).ReturnsAsync((SymbolResponseDto?)null);
        var controller = new SymbolsController(symbols.Object, Mock.Of<IMarketDataSyncService>());

        var result = await controller.GetSymbol(99);

        Assert.IsType<NotFoundObjectResult>(result.Result);
    }

    // Kontrollerer, at exchange-koden normaliseres før synkronisering.
    [Fact]
    public async Task SyncSymbols_NormalizesExchangeCodeAndCallsSyncService()
    {
        var sync = new Mock<IMarketDataSyncService>();
        var controller = new SymbolsController(Mock.Of<ISymbolService>(), sync.Object);

        var result = await controller.SyncSymbols(" okx ", CancellationToken.None);

        Assert.IsType<OkObjectResult>(result);
        sync.Verify(x => x.SyncExchangeSymbolsAsync("OKX", It.IsAny<CancellationToken>()), Times.Once);
    }

    // Kontrollerer, at statusændring på et manglende symbol returnerer Not Found.
    [Fact]
    public async Task ToggleStatus_WhenSymbolMissing_ReturnsNotFound()
    {
        var symbols = new Mock<ISymbolService>();
        symbols.Setup(x => x.UpdateSymbolStatusAsync(42, true))
            .ReturnsAsync((SymbolStatusUpdateResultDto?)null);
        var controller = new SymbolsController(symbols.Object, Mock.Of<IMarketDataSyncService>());

        var result = await controller.ToggleStatus(42, new ToggleStatusDto(true));

        Assert.IsType<NotFoundObjectResult>(result);
    }
}

```

Controllers/WatchlistsControllerTests.cs

Tester watchlist-API'et med fokus på autentificeret bruger, inputvalidering og korrekt HTTP-resultat ved tilføjelse/fjernelse.

```

using System.Security.Claims;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class WatchlistsControllerTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static WatchlistsController CreateController(IWatchlistService service, int userId = 7)
    {
        var controller = new WatchlistsController(service);
        controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext
            {
                User = new ClaimsPrincipal(new ClaimsIdentity(
                    new[] { new Claim(ClaimTypes.NameIdentifier, userId.ToString()) }, "Test"))
            }
        };
    }
}

```

```

    }
    };
    return controller;
}

// Kontrollerer, at watchlists hentes for den autentificerede bruger.
[Fact]
public async Task GetMyWatchlists_UsesAuthenticatedUserId()
{
    var service = new Mock<IWatchlistService>();
    service.Setup(x => x.GetUserWatchlistsAsync(7))
        .ReturnsAsync(new[] { new WatchlistResponseDto(1, "Main", new()) });
    var controller = CreateController(service.Object);

    var result = await controller.GetMyWatchlists();

    Assert.IsType<OkObjectResult>(result.Result);
    service.Verify(x => x.GetUserWatchlistsAsync(7), Times.Once);
}

// Kontrollerer, at en watchlist uden navn bliver afvist.
[Fact]
public async Task CreateWatchlist_BlankName_ReturnsBadRequest()
{
    var controller = CreateController(Mock.Of<IWatchlistService>());

    var result = await controller.CreateWatchlist(" ");

    Assert.IsType<BadRequestObjectResult>(result.Result);
}

// Kontrollerer, at ugyldige id-værdier bliver afvist.
[Fact]
public async Task AddSymbolToWatchlist_InvalidIds_ReturnsBadRequest()
{
    var controller = CreateController(Mock.Of<IWatchlistService>());

    var result = await controller.AddSymbolToWatchlist(new AddWatchlistItemDto(0, 1));

    Assert.IsType<BadRequestObjectResult>(result);
}

// Kontrollerer, at et mislykket service-resultat returnerer Not Found.
[Fact]
public async Task RemoveSymbolFromWatchlist_WhenServiceReturnsFalse_ReturnsNotFound()
{
    var service = new Mock<IWatchlistService>();
    service.Setup(x => x.RemoveSymbolFromWatchlistAsync(1, 10)).ReturnsAsync(false);
    var controller = CreateController(service.Object);

    var result = await controller.RemoveSymbolFromWatchlist(1, 10);

    Assert.IsType<NotFoundObjectResult>(result);
}
}

```

Helpers/FakeExchangeServiceFactory.cs

Test-double for exchange factory. Gør det muligt at kontrollere exchange-relateret controllerlogik uden at kontakte Binance eller OKX.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Helpers;

internal sealed class FakeExchangeServiceFactory : IExchangeServiceFactory
{
    private readonly HashSet<string> _supported;

    public FakeExchangeServiceFactory(params string[] supported)
    {
        _supported = supported.Select(x => x.Trim().ToUpperInvariant()).ToHashSet();
    }

    // Henter exchange.
    public IExchangeService GetExchange(string exchangeCode)
    {
        var code = exchangeCode.Trim().ToUpperInvariant();
    }
}

```

```

    if (!_supported.Contains(code))
        throw new NotSupportedException(code);

    return new FakeExchangeService(code);
}

private sealed class FakeExchangeService : IExchangeService
{
    public FakeExchangeService(string code) => ExchangeCode = code;

    public string ExchangeCode { get; }
    public DateTime HistoricalDataStartUtc => new(2017, 1, 1, 0, 0, 0, DateTimeKind.Utc);

    // Henter symbols.
    public Task<IReadOnlyList<ExchangeSymbolDto>> GetSymbolsAsync(CancellationToken cancellationToken = default) =>
        Task.FromResult<IReadOnlyList<ExchangeSymbolDto>>(Array.Empty<ExchangeSymbolDto>());

    // Henter top popular symbols.
    public Task<IReadOnlyList<string>> GetTopPopularSymbolsAsync(int topCount = 50, CancellationToken cancellationToken = default) =>
        Task.FromResult<IReadOnlyList<string>>(Array.Empty<string>());

    // Henter historical candles.
    public Task<IReadOnlyList<ExchangeCandleDto>> GetHistoricalCandlesAsync(
        string symbol, string interval, int limit = 500, long? startTime = null,
        CancellationToken cancellationToken = default) =>
        Task.FromResult<IReadOnlyList<ExchangeCandleDto>>(Array.Empty<ExchangeCandleDto>());

    // Behandler stream real time candles.
    public Task StreamRealTimeCandlesAsync(
        IReadOnlyCollection<string> symbols,
        string interval,
        Func<ExchangeCandleDto, Task> onCandleReceived,
        CancellationToken cancellationToken) => Task.CompletedTask;
}

```

Helpers/TestDbContextFactory.cs

Opretter en isoleret EF Core InMemory-database for hver test. Det gør testene uafhængige af den rigtige PostgreSQL/TimescaleDB-database.

```

using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;

namespace TradingAppTest.Helpers;

internal static class TestDbContextFactory
{
    // Opretter den relevante operation.
    public static AppDbContext Create(string? databaseName = null)
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName ?? $"TradingAppTest_{Guid.NewGuid():N}")
            .EnableSensitiveDataLogging()
            .Options;

        var context = new AppDbContext(options);
        context.Database.EnsureCreated();
        return context;
    }
}

```

Repositories/CandleRepositoryTests.cs

Tester repository-logik for candle-rækkefølge, lazy loading med endTime og update-vs-duplicate adfærd.

```

using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class CandleRepositoryTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static Candle Candle(int symbolId, DateTime openTime, decimal close) => new()
    {
        SymbolId = symbolId,
        Interval = "1m",
        OpenTime = openTime,
    }
}

```

```

    Open = close - 1,
    High = close + 1,
    Low = close - 2,
    Close = close,
    Volume = 10
};

// Kontrollerer, at de seneste candles returneres i kronologisk rækkefølge.
[Fact]
public async Task GetCandlesAsync_OneMinute_ReturnsLatestLimitInAscendingOrder()
{
    await using var db = TestDbContextFactory.Create();
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var start = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);
    for (var i = 0; i < 5; i++) db.Candles.Add(Candle(symbol.Id, start.AddMinutes(i), 100 + i));
    await db.SaveChangesAsync();

    var repo = new CandleRepository(db);
    var result = (await repo.GetCandlesAsync(symbol.Id, "1m", 3)).ToList();

    Assert.Equal(3, result.Count);
    Assert.Equal(start.AddMinutes(2), result[0].OpenTime);
    Assert.Equal(start.AddMinutes(4), result[2].OpenTime);
}

// Kontrollerer, at lazy loading kun returnerer candles før endTime.
[Fact]
public async Task GetCandlesAsync_EndTime_ReturnsOnlyCandlesBeforeEndTime()
{
    await using var db = TestDbContextFactory.Create();
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var start = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);
    for (var i = 0; i < 4; i++) db.Candles.Add(Candle(symbol.Id, start.AddMinutes(i), 100 + i));
    await db.SaveChangesAsync();
    var endMs = new DateTimeOffset(start.AddMinutes(2)).ToUnixTimeMilliseconds();

    var repo = new CandleRepository(db);
    var result = (await repo.GetCandlesAsync(symbol.Id, "1m", 100, endMs)).ToList();

    Assert.Equal(2, result.Count);
    Assert.All(result, c => Assert.True(c.OpenTime < start.AddMinutes(2)));
}

// Kontrollerer, at en eksisterende candle opdateres i stedet for at blive duplikeret.
[Fact]
public async Task AddOrUpdateCandleAsync_UpdatesExistingCandleInsteadOfDuplicating()
{
    await using var db = TestDbContextFactory.Create();
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var time = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);
    var repo = new CandleRepository(db);
    await repo.AddOrUpdateCandleAsync(Candle(symbol.Id, time, 100));

    var replacement = Candle(symbol.Id, time, 125);
    replacement.Interval = "15m"; // Repository-lagringen skal normalisere til 1m.
    await repo.AddOrUpdateCandleAsync(replacement);

    var all = db.Candles.Where(c => c.SymbolId == symbol.Id).ToList();
    var saved = Assert.Single(all);
    Assert.Equal("1m", saved.Interval);
    Assert.Equal(125, saved.Close);
}
}

```

Repositories/SymbolRepositoryTests.cs

Tester normalisering, aktive symbols, exchange-filtrering og case-insensitive symbolopslag.

```

using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class SymbolRepositoryTests

```

```
{
// Kontrollerer, at symbolfelter normaliseres før lagring.
[Fact]
public async Task AddAsync_NormalizesSymbolFields()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new SymbolRepository(db);

    var symbol = new Symbol
    {
        ExchangeId = 1,
        Name = " btcusdt ",
        BaseAsset = " btc ",
        QuoteAsset = " usdt ",
        IsActive = true
    };

    await repo.AddAsync(symbol);

    var saved = await repo.GetByIdAsync(symbol.Id);
    Assert.NotNull(saved);
    Assert.Equal("BTCUSDT", saved!.Name);
    Assert.Equal("BTC", saved.BaseAsset);
    Assert.Equal("USDT", saved.QuoteAsset);
}

// Kontrollerer, at kun aktive symboler fra aktive exchanges returneres.
[Fact]
public async Task GetAllActiveSymbolsAsync_ReturnsOnlyActiveSymbolsFromActiveExchange()
{
    await using var db = TestDbContextFactory.Create();
    db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
    db.Symbols.AddRange(
        new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true },
        new Symbol { ExchangeId = 1, Name = "ETHUSDT", BaseAsset = "ETH", QuoteAsset = "USDT", IsActive = false },
        new Symbol { ExchangeId = 2, Name = "BTC-USDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true });
    await db.SaveChangesAsync();

    var repo = new SymbolRepository(db);
    var result = (await repo.GetAllActiveSymbolsAsync()).ToList();

    var symbol = Assert.Single(result);
    Assert.Equal("BTCUSDT", symbol.Name);
    Assert.Equal("BINANCE", symbol.Exchange.Code);
}

// Kontrollerer, at symbolsøgning ikke returnerer symboler fra en anden exchange.
[Fact]
public async Task SearchAsync_WithExchangeId_DoesNotLeakSymbolsFromOtherExchange()
{
    await using var db = TestDbContextFactory.Create();
    db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = true });
    db.Symbols.AddRange(
        new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" },
        new Symbol { ExchangeId = 2, Name = "BTC-USDT", BaseAsset = "BTC", QuoteAsset = "USDT" });
    await db.SaveChangesAsync();

    var repo = new SymbolRepository(db);
    var result = (await repo.SearchAsync(2, "btc")).ToList();

    var symbol = Assert.Single(result);
    Assert.Equal(2, symbol.ExchangeId);
    Assert.Equal("BTC-USDT", symbol.Name);
}

// Kontrollerer, at opslaget er uafhængigt af store og små bogstaver.
[Fact]
public async Task GetByNameAsync_IsCaseInsensitiveAndTrimsInput()
{
    await using var db = TestDbContextFactory.Create();
    db.Symbols.Add(new Symbol
    {
        ExchangeId = 1,
        Name = "BTCUSDT",
        BaseAsset = "BTC",
        QuoteAsset = "USDT"
    });
    await db.SaveChangesAsync();

    var repo = new SymbolRepository(db);
    var result = await repo.GetByNameAsync(1, " btcusdt ");
```

```
    Assert.NotNull(result);
    Assert.Equal("BTCUSDT", result!.Name);
}
}
```

Repositories/UserRepositoryTests.cs

Tester login-opslag, soft delete, preferences-upsert og filtrering af slettede brugere.

```
using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class UserRepositoryTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static User CreateUser(string username = "ozan", string email = "ozan@test.com") => new()
    {
        Username = username,
        Email = email,
        PasswordHash = "hash",
        FirstName = "Ozan",
        LastName = "Korkmaz"
    };

    // Kontrollerer, at loginopslag virker med både brugernavn og e-mail.
    [Fact]
    public async Task GetByLoginIdentifierAsync_FindsByUsernameOrEmailCaseInsensitively()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new UserRepository(db);
        await repo.AddAsync(CreateUser());

        Assert.NotNull(await repo.GetByLoginIdentifierAsync(" OZAN "));
        Assert.NotNull(await repo.GetByLoginIdentifierAsync(" Ozan@Test.Com "));
    }

    // Kontrollerer, at sletning udføres som soft delete.
    [Fact]
    public async Task DeleteAsync_PerformsSoftDelete()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new UserRepository(db);
        var user = CreateUser();
        await repo.AddAsync(user);

        await repo.DeleteAsync(user.Id);

        // Kontroller de gemte soft-delete-flags ved at omgå det globale query-filter.
        // Adfærden for det globale filter testes separat i GetAllUsersAsync_ExcludesSoftDeletedUsers.
        var deleted = await db.Users.IgnoreQueryFilters().SingleAsync(u => u.Id == user.Id);
        Assert.True(deleted.IsDeleted);
        Assert.NotNull(deleted.DeletionRequestedAt);
    }

    // Kontrollerer, at brugerindstillinger oprettes eller opdateres uden dubletter.
    [Fact]
    public async Task SavePreferencesAsync_UpsertsPreferences()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new UserRepository(db);

        await repo.SavePreferencesAsync(new UserPreferences { UserId = "5", Settings.Json = "{\"theme\":\"dark\"}" });
        await repo.SavePreferencesAsync(new UserPreferences { UserId = "5", Settings.Json = "{\"theme\":\"light\"}" });

        var saved = await repo.GetPreferencesByUserIdAsync("5");
        Assert.NotNull(saved);
        Assert.Equal("{\"theme\":\"light\"}", saved!.Settings.Json);
        Assert.Single(db.UserPreferences);
    }

    // Kontrollerer, at sletning udføres som soft delete.
    [Fact]
    public async Task GetAllUsersAsync_ExcludesSoftDeletedUsers()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new UserRepository(db);
        var active = CreateUser("active", "active@test.com");
        var deleted = CreateUser("deleted", "deleted@test.com");
```

```
await repo.AddAsync(active);
await repo.AddAsync(deleted);
await repo.DeleteAsync(deleted.Id);

var users = (await repo.GetAllUsersAsync()).ToList();

var user = Assert.Single(users);
Assert.Equal(active.Id, user.Id);
}
}
```

Repositories/WatchlistRepositoryTests.cs

Tester indlæsning af watchlist-relationer, fjernelse af enkelte symbols og sletning af tilhørende items.

```
using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class WatchlistRepositoryTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static User User() => new()
    {
        Username = $"user_{Guid.NewGuid():N}",
        Email = $"_{Guid.NewGuid():N}@test.com",
        PasswordHash = "hash"
    };

    // Kontrollerer, at watchlisten indlæses med items, symbol og exchange.
    [Fact]
    public async Task GetWatchlistsByUserIdAsync_LoadsItemsSymbolAndExchange()
    {
        await using var db = TestDbContextFactory.Create();
        var user = User();
        db.Users.Add(user);
        var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true };
        db.Symbols.Add(symbol);
        await db.SaveChangesAsync();
        var watchlist = new Watchlist { UserId = user.Id, Name = "Crypto" };
        db.Watchlists.Add(watchlist);
        await db.SaveChangesAsync();
        db.WatchlistItems.Add(new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = symbol.Id });
        await db.SaveChangesAsync();

        var repo = new WatchlistRepository(db);
        var result = (await repo.GetWatchlistsByUserIdAsync(user.Id)).ToList();

        var loaded = Assert.Single(result);
        var item = Assert.Single(loaded.Items);
        Assert.Equal("BTCUSDT", item.Symbol.Name);
        Assert.Equal("BINANCE", item.Symbol.Exchange.Code);
    }

    // Kontrollerer, at kun det valgte symbol fjernes fra watchlisten.
    [Fact]
    public async Task RemoveItemAsync_RemovesOnlyRequestedSymbol()
    {
        await using var db = TestDbContextFactory.Create();
        var user = User();
        db.Users.Add(user);
        var s1 = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
        var s2 = new Symbol { ExchangeId = 1, Name = "ETHUSDT", BaseAsset = "ETH", QuoteAsset = "USDT" };
        db.Symbols.AddRange(s1, s2);
        await db.SaveChangesAsync();
        var watchlist = new Watchlist { UserId = user.Id, Name = "Main" };
        db.Watchlists.Add(watchlist);
        await db.SaveChangesAsync();
        db.WatchlistItems.AddRange(
            new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = s1.Id },
            new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = s2.Id });
        await db.SaveChangesAsync();

        var repo = new WatchlistRepository(db);
        await repo.RemoveItemAsync(watchlist.Id, s1.Id);

        var remaining = await db.WatchlistItems.Where(x => x.WatchlistId == watchlist.Id).ToListAsync();
        var item = Assert.Single(remaining);
        Assert.Equal(s2.Id, item.SymbolId);
    }
}
```

```

}

// Kontrollerer, at tilhørende watchlist-items fjernes sammen med watchlisten.
[Fact]
public async Task DeleteAsync_CascadeDeletesWatchlistItems()
{
    await using var db = TestDbContextFactory.Create();
    var user = User();
    db.Users.Add(user);
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var watchlist = new Watchlist { UserId = user.Id, Name = "Delete me" };
    db.Watchlists.Add(watchlist);
    await db.SaveChangesAsync();
    db.WatchlistItems.Add(new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = symbol.Id });
    await db.SaveChangesAsync();

    var repo = new WatchlistRepository(db);
    await repo.DeleteAsync(watchlist.Id);

    Assert.Null(await db.Watchlists.FindAsync(watchlist.Id));
    // EF InMemory håndhæver ikke relationelle cascade-regler. Den vigtige repository-adfærd
    // her er, at parent-posten fjernes; relationel cascade dækkes af skemaet hos den rigtige databaseprovider.
}

```

Repositories/ExchangeRepositoryTests.cs

Tester repository-logik for aktive exchanges, id-opslag, duplicate code, symbolrelationer samt oprettelse, opdatering og sletning.

```

using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

```

```

namespace TradingAppTest.Repositories;

```

```

public class ExchangeRepositoryTests

```

```

{
    // Kontrollerer, at kun aktive exchanges returneres og sorteres efter navn.
    [Fact]
    public async Task GetActiveAsync_ReturnsOnlyActiveExchangesOrderedByName()
    {
        await using var db = TestDbContextFactory.Create();
        db.Exchanges.AddRange(
            new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = true },
            new Exchange { Id = 3, Code = "KRAKEN", Name = "Kraken", IsActive = false });
        await db.SaveChangesAsync();

        var repo = new ExchangeRepository(db);
        var result = (await repo.GetActiveAsync()).ToList();

        Assert.Equal(2, result.Count);
        Assert.All(result, exchange => Assert.True(exchange.IsActive));
        Assert.Equal("BINANCE", result[0].Code);
        Assert.Equal("OKX", result[1].Code);
    }

    // Kontrollerer, at en exchange kan findes via dens id.
    [Fact]
    public async Task GetByIdAsync_ExistingExchange_ReturnsExchange()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new ExchangeRepository(db);

        var result = await repo.GetByIdAsync(1);
    }
}

```



```
    Assert.NotNull(result);
    Assert.Equal("BINANCE", result!.Code);
}

// Kontrollerer duplicate code og exclude-id ved opdatering.
[Fact]
public async Task ExistsByCodeAsync_RespectsExcludedExchangeId()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new ExchangeRepository(db);

    Assert.True(await repo.ExistsByCodeAsync("BINANCE"));
    Assert.False(await repo.ExistsByCodeAsync("BINANCE", 1));
    Assert.False(await repo.ExistsByCodeAsync("DOES_NOT_EXIST"));
}

// Kontrollerer, om en exchange har tilknyttede symbols.
[Fact]
public async Task HasSymbolsAsync_ReturnsTrueOnlyWhenSymbolsExist()
{
    await using var db = TestDbContextFactory.Create();
    db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = true });
    db.Symbols.Add(new Symbol
    {
        ExchangeId = 1,
        Name = "BTCUSDT",
        BaseAsset = "BTC",
        QuoteAsset = "USDT",
        IsActive = true
    });
    await db.SaveChangesAsync();

    var repo = new ExchangeRepository(db);

    Assert.True(await repo.HasSymbolsAsync(1));
    Assert.False(await repo.HasSymbolsAsync(2));
}

// Kontrollerer, at AddAsync gemmer en ny exchange i databasen.
[Fact]
public async Task AddAsync_PersistsExchange()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new ExchangeRepository(db);
    var exchange = new Exchange
    {
        Code = "OKX",
        Name = "OKX",
        IsActive = true
    };

    await repo.AddAsync(exchange);

    var saved = await repo.GetByIdAsync(exchange.Id);
    Assert.NotNull(saved);
    Assert.Equal("OKX", saved!.Code);
    Assert.True(saved.IsActive);
}

// Kontrollerer, at ændringer gemmes via SaveChangesAsync.
```

```
[Fact]
public async Task SaveChangesAsync_PersistsUpdatedExchange()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new ExchangeRepository(db);
    var exchange = await repo.GetByldAsync(1);
    Assert.NotNull(exchange);

    exchange!.Name = "Binance Updated";
    exchange.IsActive = false;
    await repo.SaveChangesAsync();

    var saved = await repo.GetByldAsync(1);
    Assert.NotNull(saved);
    Assert.Equal("Binance Updated", saved!.Name);
    Assert.False(saved.IsActive);
}

// Kontrollerer, at DeleteAsync fjerner en exchange uden tilknyttede data.
[Fact]
public async Task DeleteAsync_RemovesExchange()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new ExchangeRepository(db);
    var exchange = new Exchange
    {
        Code = "TEST",
        Name = "Test Exchange",
        IsActive = false
    };
    await repo.AddAsync(exchange);
    var id = exchange.Id;

    await repo.DeleteAsync(exchange);

    var deleted = await repo.GetByldAsync(id);
    Assert.Null(deleted);
}
}
```

6. Begrænsninger

EF Core InMemory er velegnet til hurtige tests af applikationslogik, men det er ikke PostgreSQL/TimescaleDB. Rå SQL, continuous aggregates og provider-specifik adfærd bør derfor suppleres med integrationstests mod en rigtig PostgreSQL/TimescaleDB testinstans. De 39 bestående tests dokumenterer repository- og controllerreglerne, men erstatter ikke system- og integrationstest.